Developing conversational applications

Generative AI

Module 3 – Lesson 2

Today's activities



- Review of lesson 1
- From Q&A to Conversation
- Customizing chat applications

Review: Strands Agents

- An open-source SDK for building and deploying AI agents
- Model-driven approach: the model is the orchestrator
  - Simple three-component architecture: model, tools, agent loop
  - Python-native and easy to get started
  - Built-in support for multiple model providers

Review: Simple Agent

Create an agent with tools and a system prompt

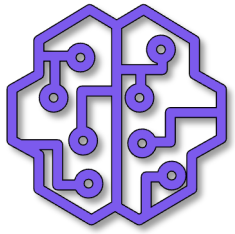
```
# Define a tool
@tool
def get_weather(location: str) -> str:
        """Get weather for a location."""
        return f"Weather in {location}: 72°F, sunny"

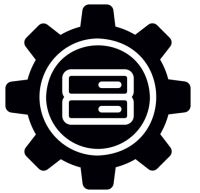
# Create agent
agent = Agent(
        model="us.anthropic.claude-3-5-sonnet-20241022-v2:0",
        tools=[get_weather]
)
result = agent("What's the weather in Paris?")

>> The weather in Paris is currently 72°F and sunny.
```

Review: Memory







Memory

Hi, my name is Andy	Hi Andy, it is nice to meet you.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you.
What is my name?	You introduced yourself as Andy.	Human: Hi, my name is Andy. AI: Hi Andy, it is nice to meet you. Human: What is my name? AI: You introduced yourself as Andy

From Q&A to Conversation

Review: Chat Models

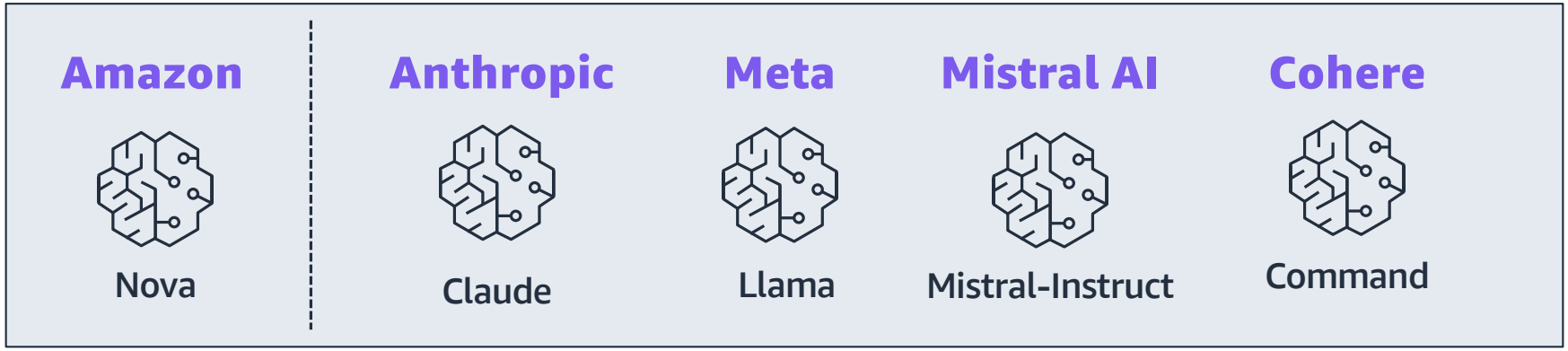
- Several LLMs are tuned and optimized for conversations
- Instruction-tuned and chat models

- Can take a sequence of messages as inputs
- System messages
- Past interactions (Human and AI messages)

- Strands Agents SDK allows you to integrate messages from several applications
  - Amazon Bedrock, custom APIs, tool integrations, etc

Examples of chat models

- Amazon Bedrock offers a suite of chat-optimized models
  - Refer to model cards for specific model details



Chat Prompt Templates

- Designed for chat models
  - » System message
  - » Human Message
  - » AI Message

```
chat_template = ChatPromptTemplate([
        ("system", "You are a helpful AI bot. Your name is {name}."),
        ("human", "Hello, how are you doing?"),
        ("ai", "I'm doing well, thanks!"),
        ("human", "{user_input}"),

])
prompt = chat_template.invoke({ "name": "Andy", "user_input": "What is your name?"})
```

Q&A vs Conversation

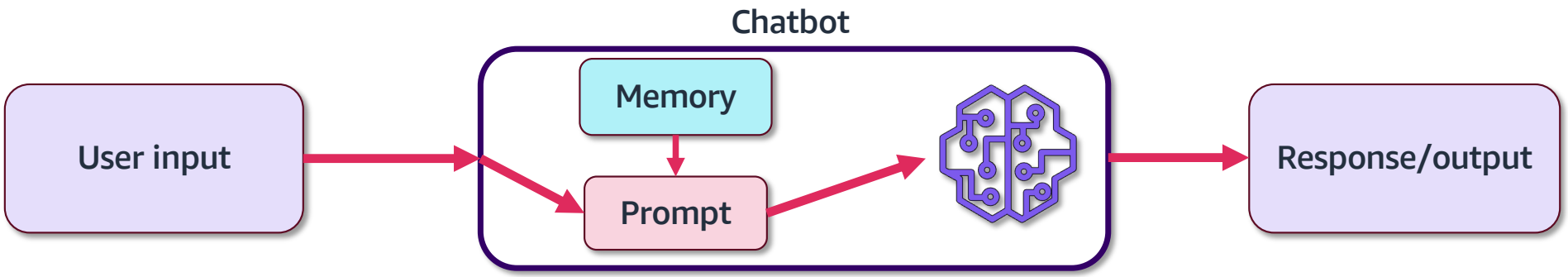
- **Question-Answering (Q&A)**: One-off interaction with the LLM
- **Conversation**: Long-running chats while maintaining the relevant context

Question – Answering (Q&A)	Conversation
Context only involves the latest prompt	Maintains a context of all past interactions
Interactions do not persist	Interactions are persisted by updating the context
No additional modules required	Typically uses a form of memory
Used for standard LLM predictions (like content creation)	Often used for chat applications (like ChatGPT)

Building a chatbot

Chatbot components

- The simplest chatbot only requires:
  - **A large language model (LLM)**
  - **Prompt template**
    - Define the role of the chatbot and general guidelines to follow
  - **Memory module**
    - Persist information across conversations



Implementing a simple chatbot

```
# Create a ConversationBufferMemory
memory = ConversationBufferMemory()

# Save context into memory
chain = prompt | llm | StrOutputParser()

# Retrieve past conversation
chat_history = memory.load_memory_variables({}).get("chat_history", "")

# Output context from memory
response = chain.invoke({
            "chat_history" : {chat_history},
            "input" : {user_input})

# Save response to memory
memory.save_context({"input": user_input}, {"output": response})
```

Customizing chat applications

Caching

- Each response generation requires substantial resources
  - Cost, compute, time

- Response caching allows retrieving answers instantly
  - Same/similar prompts can be given same responses
  - LLMs don’t need to generate a response again

- Significantly reduce inference time

Types of Caches

- **In-memory cache**:
  - Store responses in application's runtime memory
  - Fastest response time
  - Cache is lost when application runtime is reset

- **Persistent cache**:
  - Store responses offline, preferably a database
  - Persists information even when application is reset
  - More scalable solution

Response caching considerations

- Most suitable for **deterministic** queries
  - Temperature is zero
  - Creative responses might be affected by caching
- Cached information might be **outdated**
  - Set an expiration
- Be mindful of **storage requirements**
  - Use an appropriately scaled database

Issue with long conversations

- LLMs have **finite** context window
  - Number of tokens they can access while responding
- The conversation might have **redundant or obsolete** data
  - Is every message just as important?
- Longer conversations incur compounded growing costs
  - **New message + all the old messages**

Conversation Buffer Window Memory

- Maintains the last ‘k’ interactions
- Sliding window of the most recent interactions
- Prevents buffer from getting too large

```
# Create a ConversationBufferWindowMemory
memory = ConversationBufferWindowMemory(k=1)

# Save context into memory
memory.save_context({"input":"Hi"},{"output":"What's up?"})
memory.save_context({"input":"Not much. You?"},{"output":"Not
much"})

# Output context from memory
memory.load_memory_variables({})

>> {'history': 'Human:Not much. You?\nAI:Not Much'}
```

Conversation Summary Memory

- Uses an LLM to summarize the history.
  - Useful for long messages or long conversations
  - Might preserve critical information from older messages.

```
# Create a ConversationSummaryMemory
memory = ConversationSummaryMemory(llm=llm)

# Save context into memory
memory.save_context({"input":"Hi"},{"output":"What's up?"})

# Output context from memory
memory.load_memory_variables({})

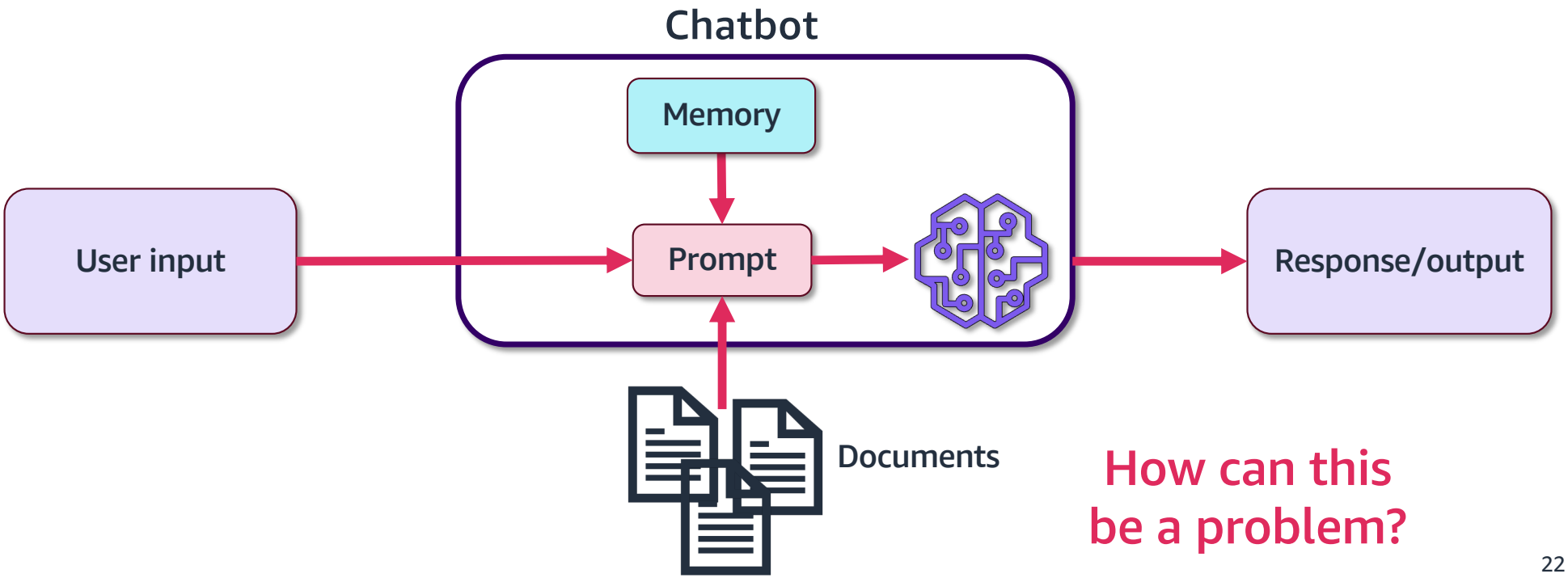
>> {'history': '\nThe human greets the AI, to which the AI
responds.'}
```

Versatility of context

- Context in the prompt can be more than just past interactions!
  - Relevant information
  - External data
  - Human feedback

Chatting with Documents

- Prompt with the entire text of the document(s)



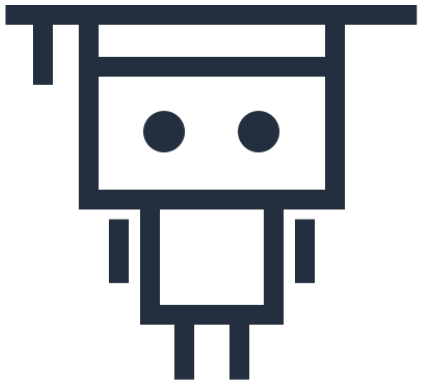
How can this be a problem?

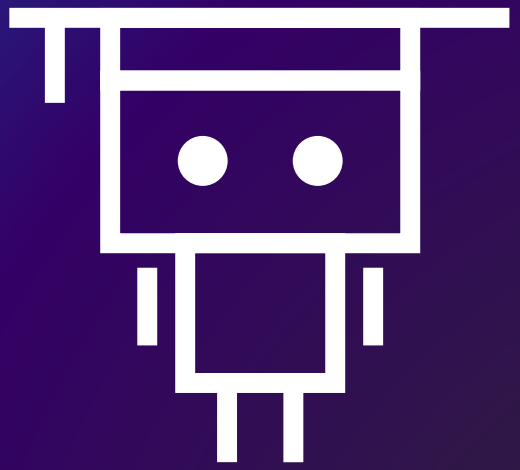
Limitations of LLMs

- **Reliability and bias**
  - Knowledge limited to the training data
  - Inability to discern false information or bias
  - **Hallucination**
- **Context window limitation**
  - Most LLMs cannot make sense of inputs exceeding their context limit
  - For instance, Nova Pro had a limit of 300k tokens at the time of release
- **Significant compute and memory costs**
  - LLMs typically require GPUs for practical applications
- **Potential copyright infringement issue**
  - May generate text similar or identical to copyrighted content

Next lesson

- This lesson showed how to develop conversational applications with Strands.
- In the next lesson, you will learn about Retrieval Augmented Generation (RAG).





Thank you!